

The Language-Resonant Oscillator — technical disclosure

Author: Prof. Dr. Benjamin Jörissen, UNESCO Chair in Digital Culture and Arts in Education (UCDCAE), Friedrich-Alexander-Universität Erlangen-Nürnberg. **Project:** akróasys — <https://github.com/joeriben/akroasys> — GPL-3.0-or-later. **This revision:** 2026-08-01.

0. Purpose of this document

This is a **defensive publication**. It describes, in ordinary technical vocabulary and in enough detail to be reproduced, the oscillator implemented in akróasys as the *Language-Resonant Oscillator* (LRO). It exists so that the system is findable and citable as prior art from its actual date of public availability, and so that the description does not depend on reading the source tree or the project's internal design documents.

It makes no patent claim and reserves none. The software is published under the GNU General Public License v3 or later; this document is published with it.

The project's own design record lives in [LCO_CONCEPT.md](#) (goal, architecture, invariants) and [plans/HANDOVER_LCO.md](#) (current implementation state), with the code-level walkthrough in [../ARCHITECTURE.md](#) §5. Those are written for contributors and use the project's own terms. This document is written for a reader outside the project — including a reader searching for prior art.

1. The problem addressed

Text-conditioned audio generation returns rendered audio. It is therefore not an oscillator: it cannot be played from a keyboard, it does not track pitch, and its output is a finished artefact rather than a signal source under continuous control.

The obvious alternative — having a language model emit synthesis code — works consistently only with very large frontier models. Small, locally runnable models produce code that is plausible in shape and wrong in substance, and they do not reliably know what a described sound *is* in synthesis terms.

The LRO addresses the second problem without a larger model, by supplying the domain knowledge the model lacks as a **curated, parametrised library of working synthesis code with measured parameter ranges and perceptual glosses**, and by letting the model — not a deterministic matcher — decide what it consults.

2. System overview

The system converts one natural-language description into one complete audio synthesis program, which is compiled and executed as the voice source of a polyphonic instrument.

user description

- (1)→ system prompt carrying the LIBRARY INDEX (entries + parameter semantics, no synthesis code)
- (2)→ CONSULTATION TURN: the model reasons about what the sound is and NAMES the library entries it wants opened
- (3)→ retrieval: exactly the named entries' real synthesis code is fetched and placed in the conversation
- (4)→ AUTHORING TURN: the model writes the synthesis program body
- (5)→ the body is wrapped in a fixed host scaffold and compiled by the real compiler of the target synthesis language
- (6)→ on failure: the compiler's own diagnostics are returned to the model as a continuation of the same conversation; repeat (4)–(6) up to a bounded number of attempts
- (7)→ control-surface derivation: parameter lines of the library's own form that survive into the written body become the player's continuous controls, and are wired into the program
- (8)→ the compiled program runs live as the instrument's oscillator, receiving per-voice gate, pitch, velocity and expression

Steps (2)–(3) are a single authoring of a single sound, not a search followed by a generation. There is **no deterministic fallback** anywhere in the path: if the model does not produce a program that compiles, the oscillator stays silent. It never substitutes a keyword-matched or default program.

The target synthesis language in the implementation is **Csound**, chosen because its opcode set already provides the domain's established methods — pulse-width modulation, FM, waveguide and modal physical models, waveshaping, granular and particle models — so that the library can consist of genuine substrate idioms rather than a reimplemented subset.

3. The library

The library is a curated data structure, not a set of presets and not a menu presented to the user. Each entry carries:

- **key** — the entry's identifier.
- **Surface forms** — the words that name it. These are *validation canon*: they are used to recognise a name in the model's own reply, and are never shown to the model and never matched against the user's prompt.
- **why** — a plain-language statement of what the entry is and what it is for. This is what the model reads in the index.
- **code** — real, working synthesis code for the target substrate, which compiles against the host scaffold of §4.
- **params** (optional) — per-parameter, a **measured** range, a default, and **named anchors each with a perceptual gloss** (for example `square 0.55 – hollow, reedy, odd harmonics only`). The ranges are measured on the shipping build, not asserted.

- `anchor_code` (optional) — the same entry rendered at each anchor of its character axis, so that the model can see which numbers correspond to which words.

Two properties of this structure carry the system's central idea:

1. **Parameters are per entry, not global.** A shared parameter vocabulary (`gritty`, `dirty`, `airy`) is required to mean something everywhere, but each entry decides which of *its own* parameters a word moves. "Membrane tension" is meaningless on an analogue oscillator; "wave" is meaningless on a drum head.
2. **The perceptual gloss is the interface between language and number.** The small model is reliable at producing anchor *words*; the anchors convert those into the numbers the substrate needs, and permit interpolation between them.

The library is generated from a curated lexicon by a build step, and a `--check` mode regenerates it in memory and fails on drift, so that the two cannot fall out of step silently.

State at the time of writing: 28 instrument entries, 51 sound-character adjectives, 17 motion entries; 11 instruments carry parameters. These counts change; the structure above is what is being disclosed.

4. The host scaffold — the only contract

The model does not write a whole program. It writes a **body**, whose single obligation is to write its output signal into one named variable (`asig`). A fixed host scaffold supplies everything else, so the model never has to get the instrument's plumbing right:

- Sample rate substituted from the running audio engine at compile time; fixed control-rate block size; one output channel per voice; unity full-scale.
- Pre-built function tables for common excitation and transfer needs (sine, cosine, a Chebyshev transfer function, a strike impulse).
- **One numeric instrument definition with a voice index as its parameter, and N always-on instances** (N = the polyphony ceiling). There is no score event per note. A note is an *edge* on a per-voice trigger channel.
- **Per-voice control channels** — gate, frequency, velocity, pressure, timbre, trigger — read by the body at control rate; the gate is de-clicked with a portamento filter and the frequency is range-limited.
- A per-voice **time-since-this-note-began** variable, reset on the trigger edge, for anything the body wants to shape over the note's lifetime.
- A fixed output tail: gate multiplication, expression gain, a headroom factor and a soft clip.

The consequence, and it is deliberate: the body is a **spectrum source**. The instrument's own amplitude envelope, filter, glide and expression belong to the synthesizer, and a body that brought its own redundant amplitude envelope would take the player's away. Every struck idiom in the library is therefore continuously excited rather than one-shot.

5. The compile-verify-repair loop

The wrapped program is compiled by the **real compiler of the target language** — the same compiler that will run it — either as a syntax-check invocation of the command-line binary or through the shared library. The environment is checked for a usable compiler *before* any inference is spent, since without one every attempt would fail identically.

On a compile failure the loop does not re-author from nothing. It returns to the model, as a continuation of the same conversation: the original request, the model's own failed reply, and **every distinct diagnostic seen so far** — not only the latest. The model therefore edits what it wrote. The loop runs to a bounded attempt ceiling (default 6) and stops early if a reply repeats without producing a new diagnostic.

Diagnostics are additionally translated into mechanical hints where the failure is a known substrate idiom error, so that the model is corrected about the substrate rather than about its intention.

6. Control-surface derivation from the generated program

The player's continuous controls for a generated instrument are **derived from the generated source itself**, not declared by the model and not fixed in advance.

- A control exists if and only if a **parameter line of the library's own documented shape** stands in the written body — that is, the model copied a parameter line out of an entry it was shown. Its **name and range come from the library**, so the panel can only ever show the library's vocabulary; the model's only say is the *number*, i.e. where the control starts.
- A line the body never reads does not become a control.
- The derivation is a function of the **body alone**. What the consultation happened to open is deliberately not consulted, so that recompiling an unedited body yields the same controls.
- The model never wires anything. A separate step rewrites each such line's literal number into an expression over the corresponding host control channel, changing only the number and leaving the line's comment and line count intact, so that compiler diagnostics still map back onto what the model wrote.

The last point is the reason this is done by derivation rather than by asking the model for a declaration: a model that writes a correct declaration and then leaves its own fixed numbers in the code produces a control that moves nothing. That failure was observed, and is what this construction removes.

7. Runtime execution and hot swap

The compiled program is executed live by a wrapper around one instance of the synthesis engine, exposing prepare / set-voice-controls / render / read-voice-buffer. The host holds **two** engine instances so that a newly authored program can be compiled on a background thread while the previously active one keeps sounding, and the swap happens in the synthesis path only. Voices are always running; a note-on is a trigger edge and a gate change on that voice's channels.

8. What is deliberately absent

These are distinguishing features, and each was arrived at by rejecting the alternative:

- **No deterministic selection layer between the user's words and the library.** An earlier implementation compared the user's words against the library's surface forms and quoted the entries that matched. It was removed: a word matcher decides for the model what it is allowed to see, and has a word matcher's holes. The model names what it wants; the code recognises names.
- **No enumerated timbre selector.** No waveform switch, no instrument menu, no fixed set of permitted results ahead of the prompt.
- **No fallback oscillator.** Without the language model there is no oscillator.
- **No runtime machinery outside the emitted program.** There is no frame store, no wavetable bake, no capture buffer, no transport wrapped around the generated code. Anything the sound does, including any morphing or motion, is expressed in the emitted source.
- **Free combination is the point.** The library is not a palette of engines with a selector over it. Any number of synthesis methods can be wired together in one emitted program, which is precisely what no practical control surface could offer.

9. Record of public disclosure

Published in the public repository <https://github.com/joeriben/akroasys> (created 2026-03-28), on the default branch. The dates below are the commit dates on that branch:

Date	Commit	Element first publicly available
2026-07-16	<code>3d7663a4</code>	Live Csound engine wrapper in the audio path
2026-07-22	<code>3728a42f</code>	The language model writes the Csound orchestra; library-consulted authoring (<code>backend/lco_write.py</code>)
2026-07-24	<code>e79031ae</code>	The two-turn consultation: the model names what it wants opened
2026-07-25	<code>f394b5a9</code>	Public prose description of the oscillator in <code>README.md</code>
2026-08-01	<code>a8b45824</code> , <code>0e7f9709</code>	Control-surface derivation of §6: a library parameter surviving into the written body becomes a player-facing control
2026-08-01	<code>8c36dc13</code>	Per-part level controls derived the same way
2026-08-01	<code>7f2b3d42</code>	The negative case: a parameter line the body never reads does not become a control

The repository has been public since its creation on 2026-03-28 and every commit above was pushed to its default branch. Commit dates are settable by the author; the load-bearing evidence is GitHub's own push record for this repository, and the third-party archive copies listed below.

Reference implementation, for enablement: `backend/lco_write.py` (library rendering, consultation, authoring, scaffold, compile-repair loop, control derivation and wiring), `backend/lco_library.json` and `backend/dco_lexicon.json` (the library), `src/dsp/CsoundEngine.{h,cpp}` (live execution), and `src/inference/PipeInference.cpp` with `docs/IPC_PROTOCOL.md` (the transport between the plugin and the model process).

Third-party archive copies

These exist so the date does not rest on the repository alone.

Archive	Identifier	Status
Zenodo (DOI, via the GitHub–Zenodo integration on a tagged release)	—	to be issued with v3.0.0
Software Heritage (permanent SWHID for the archived source)	—	to be requested

When each is issued, its identifier belongs in this table and in `CITATION.cff`.